Heart Disease Risk Prediction

End-to-End MLOps: Model Development, CI/CD and Production Deployment

Course: Machine Learning Operations (AIMLCZG523) - Assignment 01

Student name / ID: <insert name and BITS ID>

Code repository: <insert GitHub repository URL>

Deployed API: <insert URL, or: local Minikube - see Section 9>

1. Summary

This project delivers a reproducible machine learning solution that predicts the presence of heart disease from 13 clinical features. The solution covers data acquisition, exploratory analysis, two tuned classifiers, MLflow experiment tracking, a packaged scikit-learn pipeline, a tested CI workflow, a Docker image serving a FastAPI endpoint, Kubernetes manifests, and Prometheus and Grafana monitoring.

Item	Result
Dataset	UCI Heart Disease (Cleveland), 303 rows, 302 after removing one duplicate
Selected model	Random Forest (300 trees, min_samples_leaf 3), chosen by highest mean cross-validated ROC-AUC
Cross-validated ROC-AUC	0.921 +/- 0.035 (5-fold, training split)
Hold-out test (61 patients)	Accuracy 0.770, precision 0.769, recall 0.714, ROC-AUC 0.887
Tests	19 unit tests passing; flake8 clean
Serving	FastAPI: /predict, /health, /metrics; 17 to 26 ms per prediction in local tests

Scope note. Items executed and verified in the development environment are: data download, EDA, training, MLflow tracking, unit tests, lint, a clean virtual-environment install, and live API calls. Items that require the student environment are marked with a yellow INSERT SCREENSHOT box: GitHub Actions runs, Docker build and run, Kubernetes deployment, and the Grafana dashboard. The Dockerfile, manifests, workflow, and dashboard are provided and syntax-validated.

2. Setup and Installation

Requirements: Python 3.11 or later, Git, Docker, and optionally Minikube and kubectl.

```
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
python scripts/download_data.py
export PYTHONPATH=src
python -m heartml.eda        # cleaning and figures
python -m heartml.train      # tuning, CV, MLflow logging, model packaging
pytest -v && flake8 src tests scripts
```

The download script tries the UCI repository first and falls back to a GitHub mirror of the cleaned CSV when UCI is unreachable. A clean virtual-environment install from requirements-dev.txt was verified: lint passes and all 19 tests pass.

3. Data Acquisition and Exploratory Data Analysis

3.1 Dataset and cleaning

The dataset contains 13 predictors and one target. Numeric features are age, resting blood pressure (trestbps), cholesterol (chol), maximum heart rate (thalach) and ST depression (oldpeak). Categorical features are sex, chest pain type (cp), fasting blood sugar (fbs), resting ECG (restecg), exercise-induced angina (exang), ST slope (slope), number of major vessels (ca) and thalassemia (thal).

- Duplicates: 1 duplicate row removed, leaving 302 rows.
- Missing values: none present in the source file used. The pipeline still imputes (median for numeric, most frequent for categorical) so that raw UCI files containing '?' values are handled without changes.
- Target: binarised to 0 (absence) and 1 (presence). UCI severity values 1 to 4 map to 1.
- Label alignment: the GitHub mirror stores an inverted target relative to UCI. The download script flips it so that 1 means disease. Evidence: before the flip, exang, oldpeak and ca correlated negatively with the target and thalach positively, which is the pattern of healthy patients; after the flip the signs are clinically consistent.

3.2 Class balance

The classes are close to balanced: 164 patients without disease (54.3%) and 138 with disease (45.7%). Accuracy is therefore a valid metric, and recall is tracked because missed disease cases carry the highest cost.

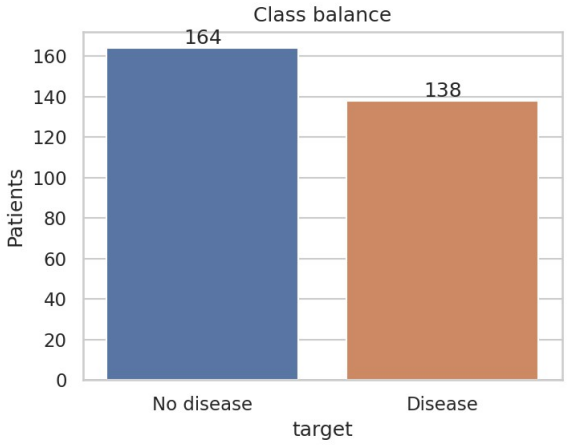


Figure 1. Class balance after cleaning

3.3 Distributions and relationships

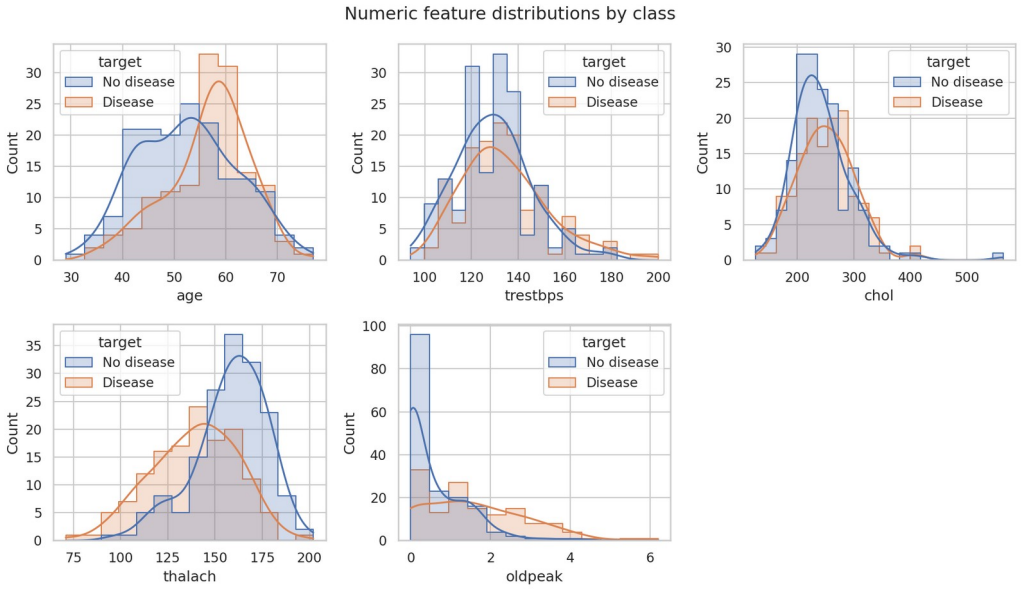


Figure 2. Numeric feature distributions by class

Patients with disease tend to show a lower maximum heart rate and higher ST depression. Age and resting blood pressure show weaker separation. Cholesterol has a long right tail with four values above 400 mg/dl (maximum 564).

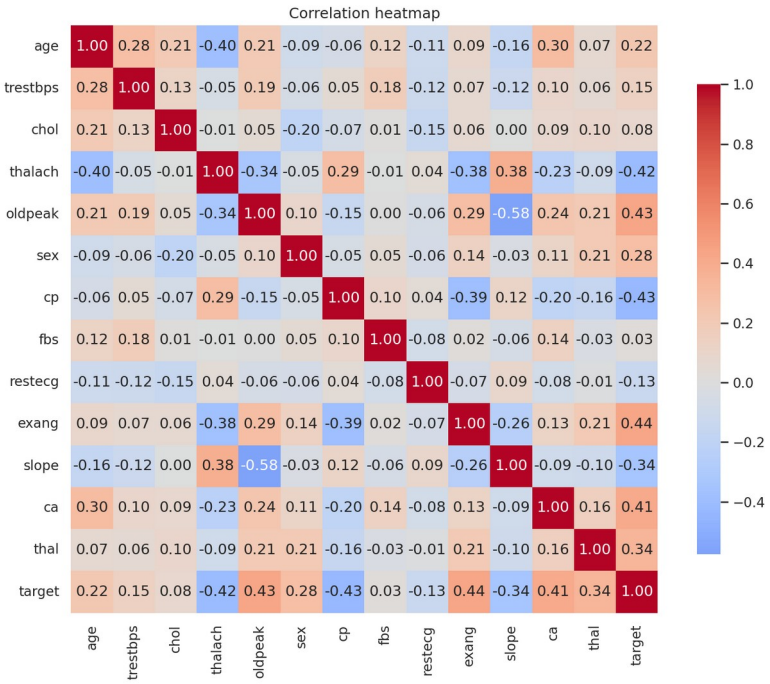


Figure 3. Correlation heatmap

The strongest correlations with the target are exercise-induced angina (+0.44), chest pain type (-0.43), ST depression (+0.43), maximum heart rate (-0.42) and number of major vessels (+0.41). No pair of predictors shows correlation high enough to require removal.

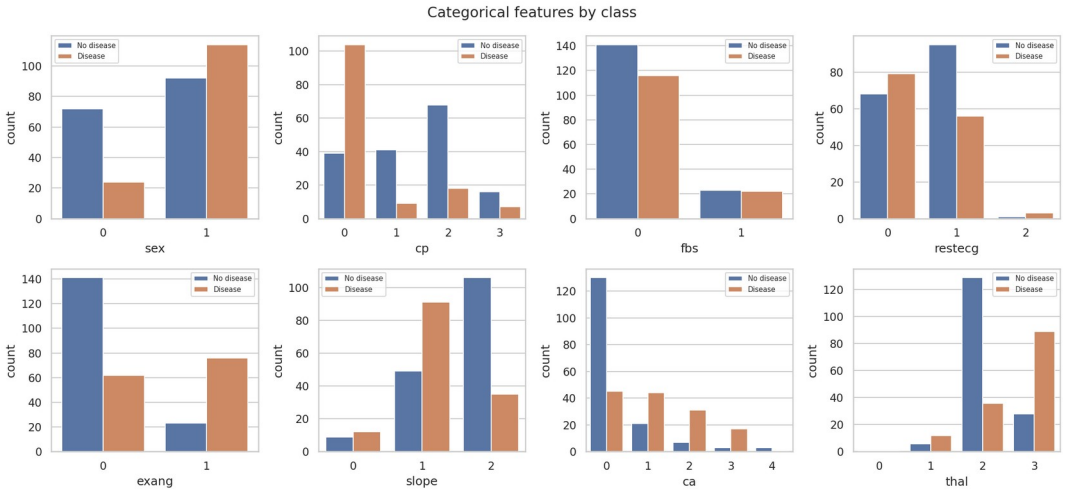


Figure 4. Categorical features by class

4. Feature Engineering and Model Development

4.1 Preprocessing pipeline

All preprocessing lives inside one scikit-learn Pipeline so that training and serving apply identical transformations.

Feature group	Steps	Rationale
Numeric (5)	Median imputation, StandardScaler	Scale-sensitive for logistic regression; robust to outliers via median
Categorical (8)	Most-frequent imputation, OneHotEncoder(handle_unknown='ignore')	Codes are nominal; unseen categories at inference do not raise errors

4.2 Models, tuning and evaluation protocol

Data was split 80/20 with stratification (241 training, 61 test rows, random_state 42). Each model was tuned with GridSearchCV using stratified 5-fold cross-validation and ROC-AUC as the objective. The tuned model was then re-evaluated with cross_validate on five metrics. The test split was used once, for final reporting, and not for selection.

Model	Search grid	Best parameters
Logistic Regression	C in {0.01, 0.1, 1, 10}, L2 penalty (4 candidates)	C = 0.1
Random Forest	n_estimators {100, 300}, max_depth {3, 5, None}, min_samples_leaf {1, 3} (12 candidates)	300 trees, max_depth None, min_samples_leaf 3

Metric (mean +/- std, 5-fold CV)	Logistic Reg.	Random Forest	LR test	RF test
Accuracy	0.851 +/- 0.021	0.838 +/- 0.040	0.820	0.770
Precision	0.859 +/- 0.069	0.830 +/- 0.082	0.815	0.769
Recall	0.818 +/- 0.076	0.827 +/- 0.067	0.786	0.714

Metric (mean +/- std, 5-fold CV)	Logistic Reg.	Random Forest	LR test	RF test
F1	0.833 +/- 0.024	0.824 +/- 0.039	0.800	0.741
ROC-AUC	0.915 +/- 0.039	0.921 +/- 0.035	0.921	0.887

4.3 Model selection and interpretation

The selection rule was fixed before training: highest mean cross-validated ROC-AUC. Random Forest wins by 0.006, which is far smaller than the fold-to-fold standard deviation of about 0.035. The two models cannot be separated on this dataset given that variability. On the 61-patient hold-out set, Logistic Regression scores higher (ROC-AUC 0.921 against 0.887), but a test set of this size carries wide uncertainty, so the preset rule was retained to avoid selecting on test data.

Recommendation: for a clinical deployment, retrain on the full dataset and compare the models with repeated cross-validation or a larger external cohort. Logistic Regression is the simpler and more interpretable alternative and remains a valid choice.

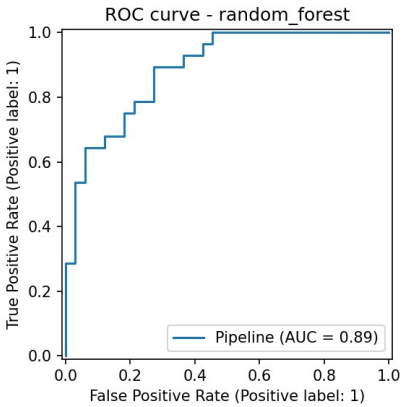


Figure 5. ROC curve, Random Forest (test split)

5. Experiment Tracking with MLflow

Every training execution logs to an MLflow experiment named heart-disease-classification, backed by SQLite (mlflow.db) with a local artifact store (mlartifacts/). Three runs are recorded per execution: one per model and one for the selected model.

Logged item	Content
Parameters	Best hyper-parameters, model name, CV folds, test size, random state, train and test sizes
Metrics	CV mean and std for accuracy, precision, recall, F1, ROC-AUC; test accuracy, precision, recall, F1, ROC-AUC
Artifacts	ROC curve, confusion matrix, grid_search_results.csv, serialized model with signature and input example, model metadata JSON

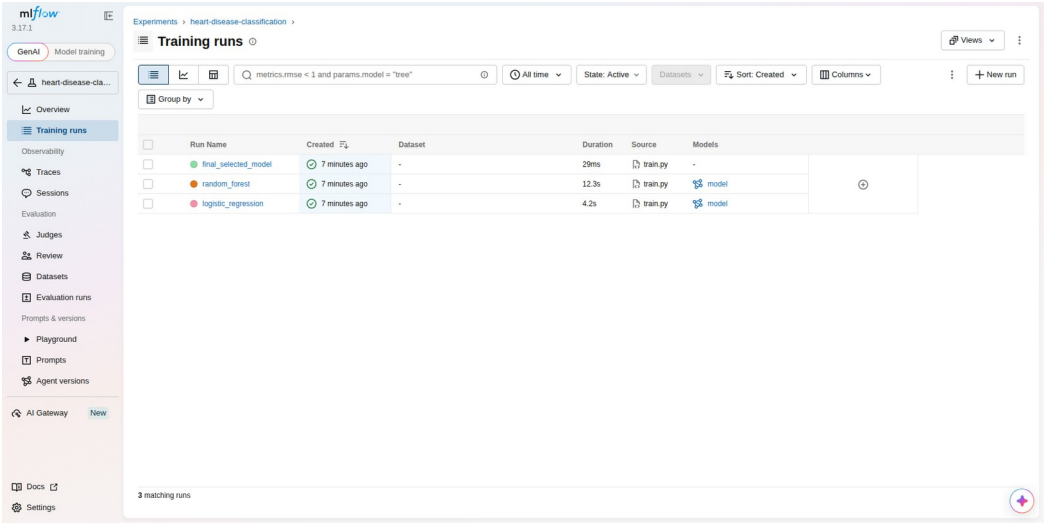


Figure 6. MLflow experiment: three tracked runs, each with a logged model

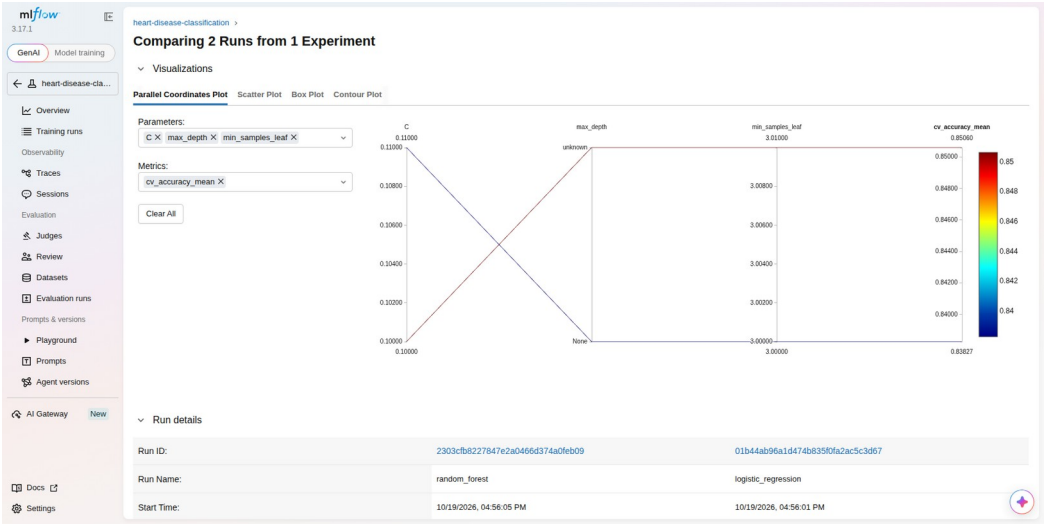


Figure 7. MLflow run comparison: Random Forest and Logistic Regression

Start the interface with: mlflow ui --backend-store-uri sqlite:///mlflow.db. If the interface renders a blank error page in an environment with a non-standard locale, start the server with --gunicorn-opts "--workers 1".

6. Model Packaging and Reproducibility

- **Model format:** the full Pipeline (preprocessing plus classifier) is saved with joblib as models/model.joblib (1.3 MB) and also logged to MLflow with cloudpickle serialization. A metadata file records the selected model, metrics, feature list and library versions.
- **Dependencies:** requirements.txt (training), requirements-dev.txt (testing and lint), and requirements-serve.txt (runtime, exact version pins matching the training environment: scikit-learn 1.8.0, numpy 2.4.4, pandas 3.0.2). Pinning serving versions prevents pickle incompatibility.
- **Determinism:** fixed random_state 42 for the split, cross-validation and models. Preprocessing has a determinism unit test.
- **Clean setup:** a fresh virtual environment built from requirements-dev.txt passed flake8 and all 19 tests.

7. CI/CD Pipeline and Automated Testing

7.1 Unit tests

File	Tests	Coverage of behavior
tests/test_data.py	6	Target binarisation, duplicate removal, bad values to NaN, column order, schema validation, real file sanity
tests/test_features.py	4	Finite and scaled output, imputation, unseen category handling, determinism
tests/test_model.py	4	Both models fit and return valid probabilities, metric set, quality floor (ROC-AUC above 0.8) on the saved model
tests/test_api.py	5	Health, prediction schema and confidence range, missing field (422), out-of-range value (422), metrics exposure

Result: 19 passed. Line coverage of the source package is 57% overall (API 92%, features 100%, data 90%); the training and EDA modules are exercised end to end by the CI training step rather than by unit tests. Raising unit coverage of train.py is a documented improvement.

7.2 GitHub Actions workflow

The workflow .github/workflows/ci.yml runs on every push to main and on pull requests. Any failing step stops the pipeline and the logs show the failing command.

#	Step	Behavior
1	Install	pip install -r requirements-dev.txt with pip cache
2	Lint	flake8 src tests scripts (max line length 100); fails on violation
3	Dataset	python scripts/download_data.py
4	Unit tests	pytest with coverage and JUnit XML
5	Train	python -m heartml.train, output captured with tee
6	Artifacts	Uploads junit.xml, coverage.xml, train_output.txt, models/, mlartifacts/, mlflow.db (runs even on failure)
7	Docker job	Builds the image, starts the container, calls /health and /predict, asserts a valid prediction, prints container logs

INSERT SCREENSHOT: GitHub Actions run page showing both jobs green, plus the Artifacts section (Actions tab, latest run).

INSERT SCREENSHOT: A failing run (for example, introduce a lint error on a branch) showing that the pipeline fails with a clear log.

Local evidence of the same steps is stored in the evidence folder (flake8.txt, pytest_output.txt, train_output.txt).

8. Model Containerization

The Dockerfile uses python:3.11-slim, installs only the pinned serving dependencies, copies the package and the packaged model, runs as a non-root user, and defines a HEALTHCHECK on /health.

```
docker build -t heart-api:1.0.0 .
docker run -p 8000:8000 heart-api:1.0.0
```

8.1 API contract

Endpoint	Description
POST /predict	Accepts JSON with the 13 features (validated ranges, 422 on invalid input). Returns prediction (0 or 1), label, confidence (probability of the predicted class), probability_disease and model_version.
GET /health	Returns 200 with model name and version when the model is loaded, 503 otherwise. Used by Docker and Kubernetes probes.
GET /metrics	Prometheus exposition format.

Verified request and response (executed against the running API with uvicorn; the container runs the same application code):

```
POST /predict
{"age":63,"sex":1,"cp":3,"trestbps":145,"chol":233,"fbs":1,"restecg":0,
             "thalach":150,"exang":0,"oldpeak":2.3,"slope":0,"ca":0,"thal":1}
200 {"prediction":0,"label":"no heart disease","confidence":0.673,
     "probability_disease":0.327,"model_version":"random_forest"}
POST /predict {"age":63}   ->   422 (validation error)
```

INSERT SCREENSHOT: Terminal output of docker build, docker run, and curl against http://localhost:8000/predict from the container.

9. Production Deployment

Kubernetes manifests in k8s/ deploy two replicas with readiness and liveness probes on /health, resource requests and limits, and Prometheus scrape annotations. A Service of type LoadBalancer exposes port 80 to container port 8000. An optional Ingress routes heart-api.local through the NGINX ingress controller.

Step	Command
Start cluster	minikube start
Build image in cluster	eval $(minikube docker-env) && docker build -t heart-api:1.0.0 .
Deploy	kubectl apply -f k8s/deployment.yaml -f k8s/service.yaml
Expose LoadBalancer	minikube tunnel (separate terminal), then kubectl get svc heart-api
Verify	curl http://<EXTERNAL-IP>/health and POST to /predict
Optional Ingress	minikube addons enable ingress && kubectl apply -f k8s/ingress.yaml

INSERT SCREENSHOT: kubectl get pods,svc,deploy output with both pods Running and the service EXTERNAL-IP assigned.

INSERT SCREENSHOT: Successful curl /health and /predict through the LoadBalancer address.

10. Monitoring and Logging

10.1 Request logging

Every request produces one structured JSON log line with request id, method, path, status, latency in milliseconds and client address. Each prediction also logs the predicted class and confidence. Logs go to stdout (visible with docker logs and kubectl logs), and a rotating file is enabled with the LOG_FILE variable. Captured output:

```
{"event": "request", "request_id": "29d2adb0", "method": "POST", "path":
"/predict",
  "status": 200, "latency_ms": 25.62, "client": "127.0.0.1"}
{"event": "prediction", "prediction": 0, "confidence": 0.9741}
{"event": "request", "request_id": "6d15e021", "method": "POST", "path":
"/predict",
  "status": 422, "latency_ms": 0.57, "client": "127.0.0.1"}
```

10.2 Prometheus and Grafana

Metric	Purpose
api_requests_total{method,path,status}	Request volume and error rate
api_request_latency_seconds (histogram)	Latency percentiles per path
model_predictions_total{prediction}	Class mix of predictions; shifts indicate input drift
model_confidence (histogram)	Distribution of prediction confidence
model_loaded	Model availability gauge

docker compose up --build starts the API, Prometheus (port 9090) and Grafana (port 3000). Grafana provisions the Prometheus data source and the Heart Disease API dashboard automatically. The dashboard has five panels: request rate by path and status, p95 latency, predictions by class, model loaded, and error rate. Verified locally: the /metrics endpoint returned api_requests_total, model_predictions_total and model_loaded with correct counts after test calls.

INSERT SCREENSHOT: Grafana dashboard 'Heart Disease API' after sending 20 or more requests (use a loop of curl calls).

INSERT SCREENSHOT: Prometheus targets page showing the heart-api target in state UP.

11. Architecture

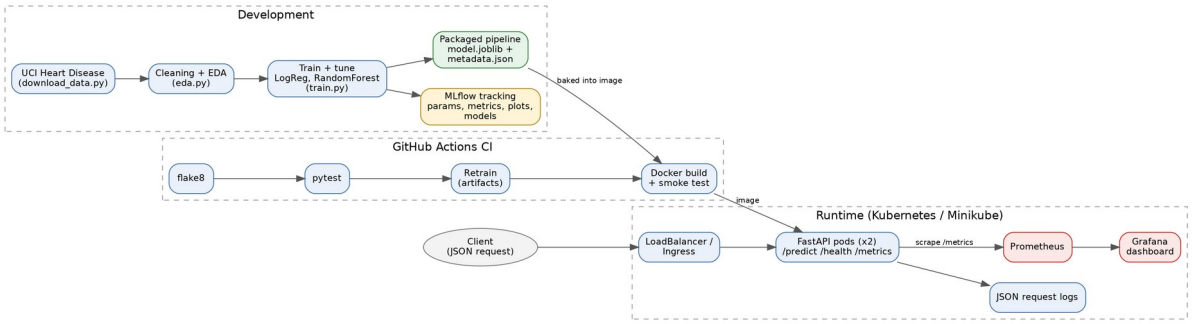


Figure 8. System architecture: development, CI, and runtime

Flow: data is downloaded and cleaned, models are tuned and tracked in MLflow, and the selected pipeline is saved. CI lints, tests, retrains and builds the image. The image runs as two Kubernetes pods behind a LoadBalancer. Prometheus scrapes /metrics and Grafana visualises the data.

12. Limitations and Recommended Improvements

- Small dataset (302 patients, 61 in the test split): metric estimates are noisy and the two models cannot be separated reliably. Recommend repeated cross-validation and an external validation set.
- The dataset is not representative of a general population and the tool is for educational purposes only; it is not a medical device.
- Source labels: the mirror used in the development environment required a label flip; recommend running scripts/download_data.py --source uci in the student environment, where UCI is reachable, and re-running training. The processed UCI file encodes slope and thal with different integer codes, which the pipeline treats as nominal categories and therefore handles without change.
- Model registry and promotion: the MLflow model registry is not used. Recommend registering the selected model and promoting by stage in a multi-environment setup.
- Test coverage of train.py is low (34%). Recommend adding tests with a small synthetic dataset for run_training.
- Monitoring covers service health and prediction mix. Recommend adding input drift statistics and alert rules in Prometheus.

13. Deliverables Checklist

Deliverable	Location	Status
Code, Dockerfile, requirements	Repository root, src/, requirements*.txt	Complete
Cleaned dataset and download script	data/processed/, scripts/download_data.py	Complete
EDA, training and inference scripts	src/heartml/ (eda, train, api)	Complete
Unit tests	tests/	Complete (19 passing)
GitHub Actions workflow	.github/workflows/ci.yml	Provided; run to capture screenshots
Deployment manifests	k8s/, docker-compose.yml, monitoring/	Provided; deploy to capture screenshots
Screenshots	screenshots/	MLflow done; CI, Docker, K8s, Grafana to add
Short video of the pipeline	Student recording	To record (see README run order)
Deployed API URL or local access instructions	Section 9	Local Minikube instructions provided